



POLITECNICO
MILANO 1863

Kafka

Scaling real-time data stream handling

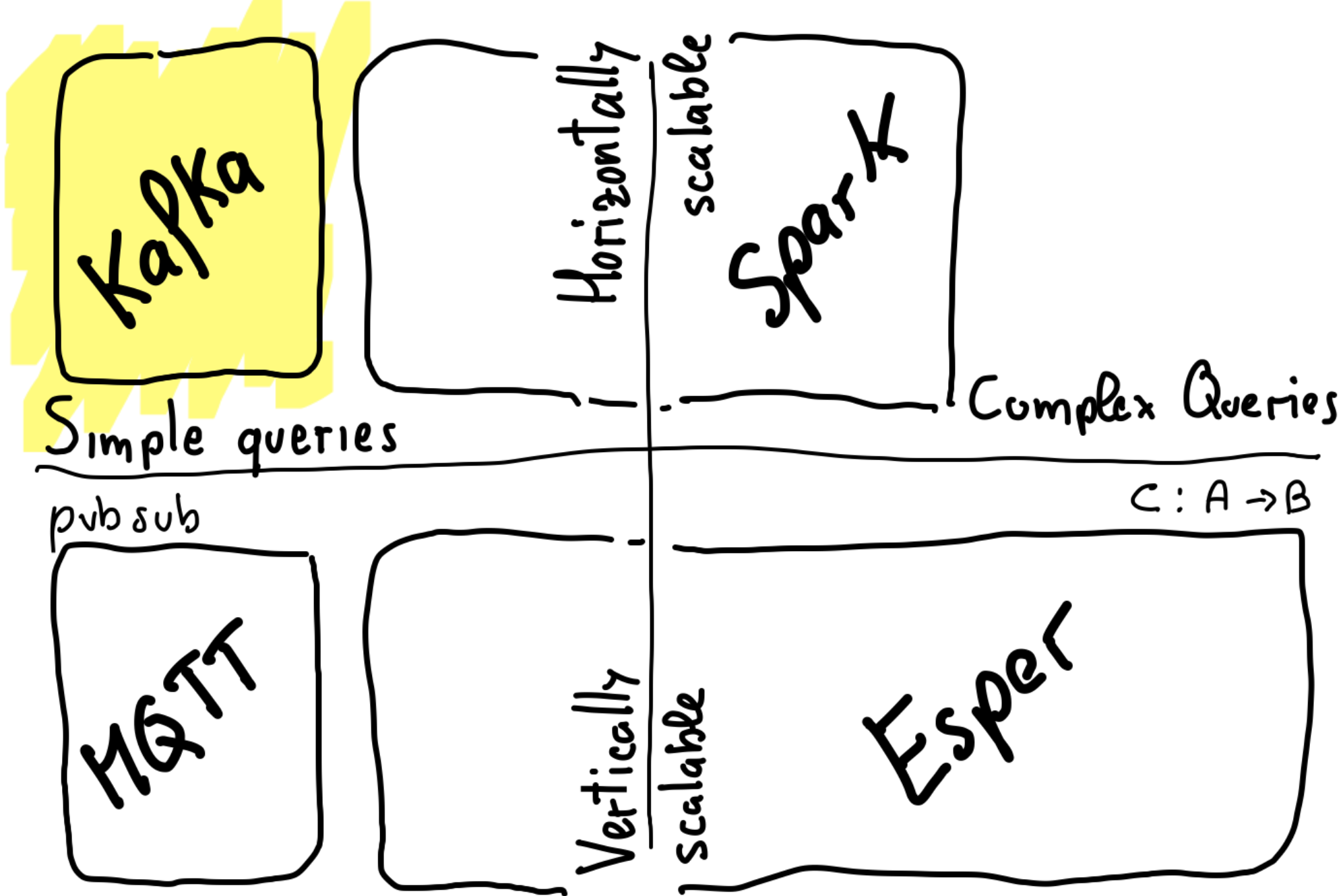
Emanuele Della Valle

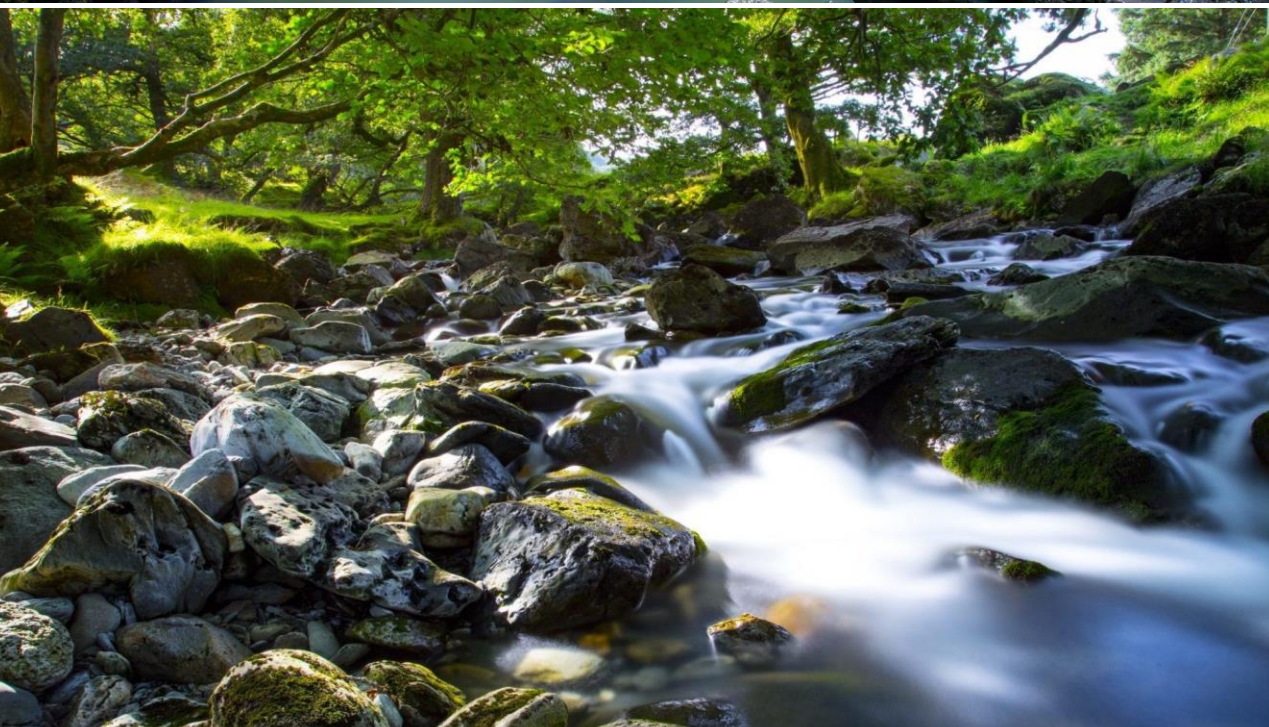
emanuele.dellavalle@polimi.it



POLITECNICO
MILANO 1863

Positioning this lecture





Throughput Latency Data/Message

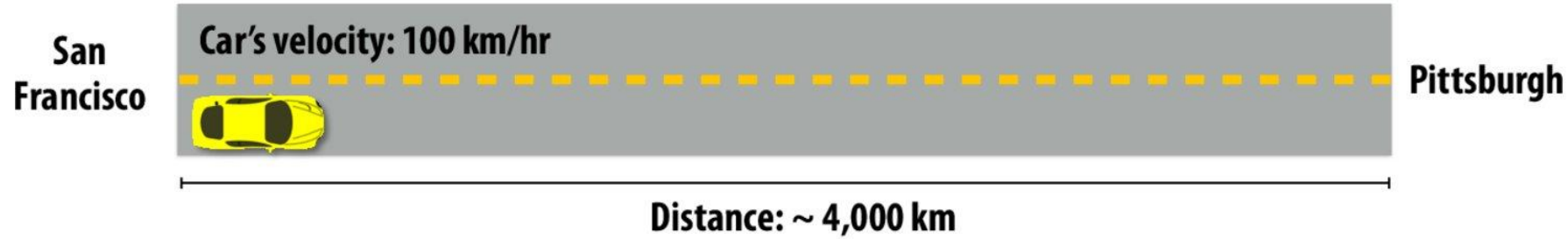
Few important concepts

Everyone wants to get to Pittsburgh!

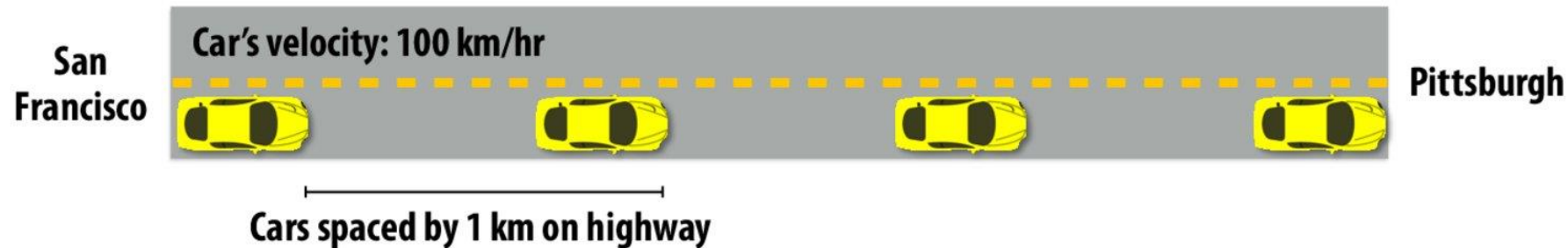
(Latency vs. throughput review)



POLITECNICO
MILANO 1863



Latency of moving a person from San Francisco to Pittsburgh: 40 hours

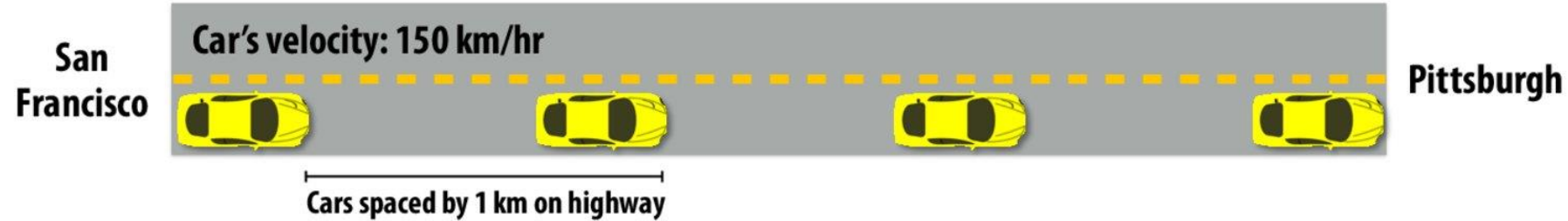


Throughput: 100 people per hour (1 car every 1/100 of an hour)

Improving throughput

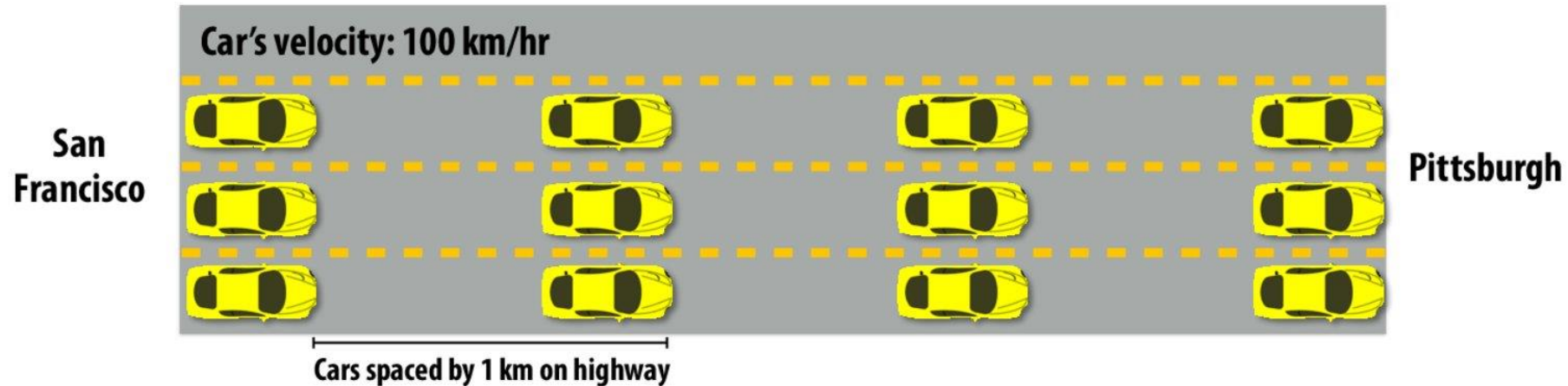


POLITECNICO
MILANO 1863



Approach 1: drive faster!

Throughput = 150 people per hour (1 car every 1/150 of an hour)



Approach 2: build more lanes!

Throughput: 300 people per hour (3 cars every 1/100 of an hour)

Review: latency vs throughput



POLITECNICO
MILANO 1863

Latency

The amount of time needed for an operation to complete.

A memory load that misses the cache has a latency of 200 cycles

A packet takes 20 ms to be sent from my computer to Google

Asking a question on Piazza gets response in 10 minutes

throughput

The rate at which operations are performed.

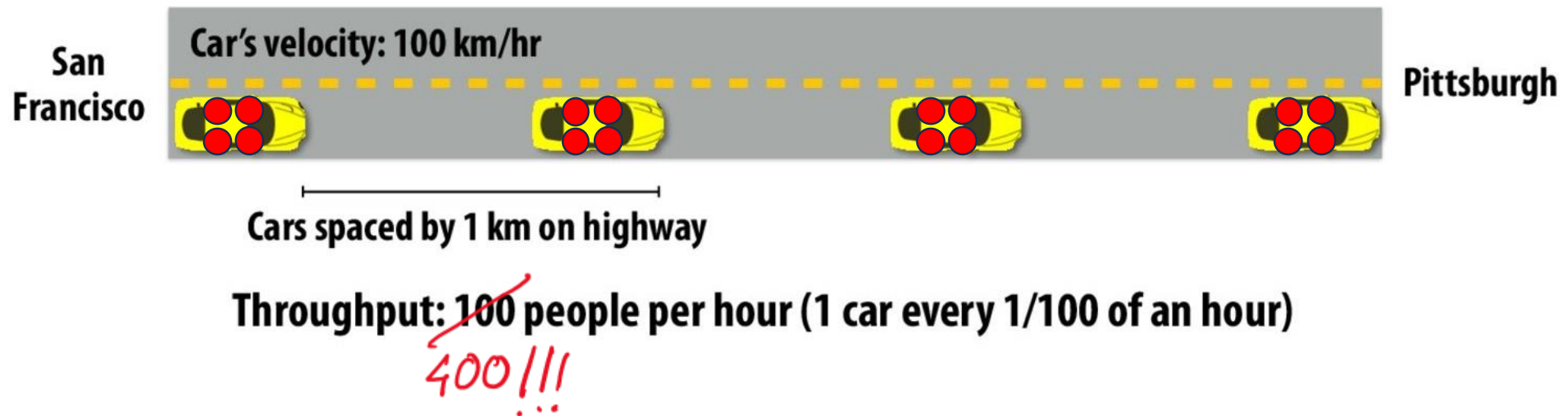
Memory can provide data to the processor at 25 GB/sec.

A communication link can send 10 million messages per second

The TAs answer 50 questions per day on Piazza

One more thought

- What about changing the number of car occupants?



space required to transport 60 people



POLITECNICO
MILANO 1863



car

text/json



bus

auto-gen binary (AVRO)



bicycle

manually gen binary

(Poster in city of Muenster Planning Office, August 2001) Credit: PressOffice City of Munster, Germany



Data/Message matters!

- Fixed the size of a message (i.e., the space between two cars)
- The more people fit in the car, the higher the throughput
- The larger the data points per message, the higher the throughput
- **Compression** and **binary encoding** are the typical ways to increase data per message

Taming
*Myriads of tiny flows
that you can collect*
with
Event-based Systems



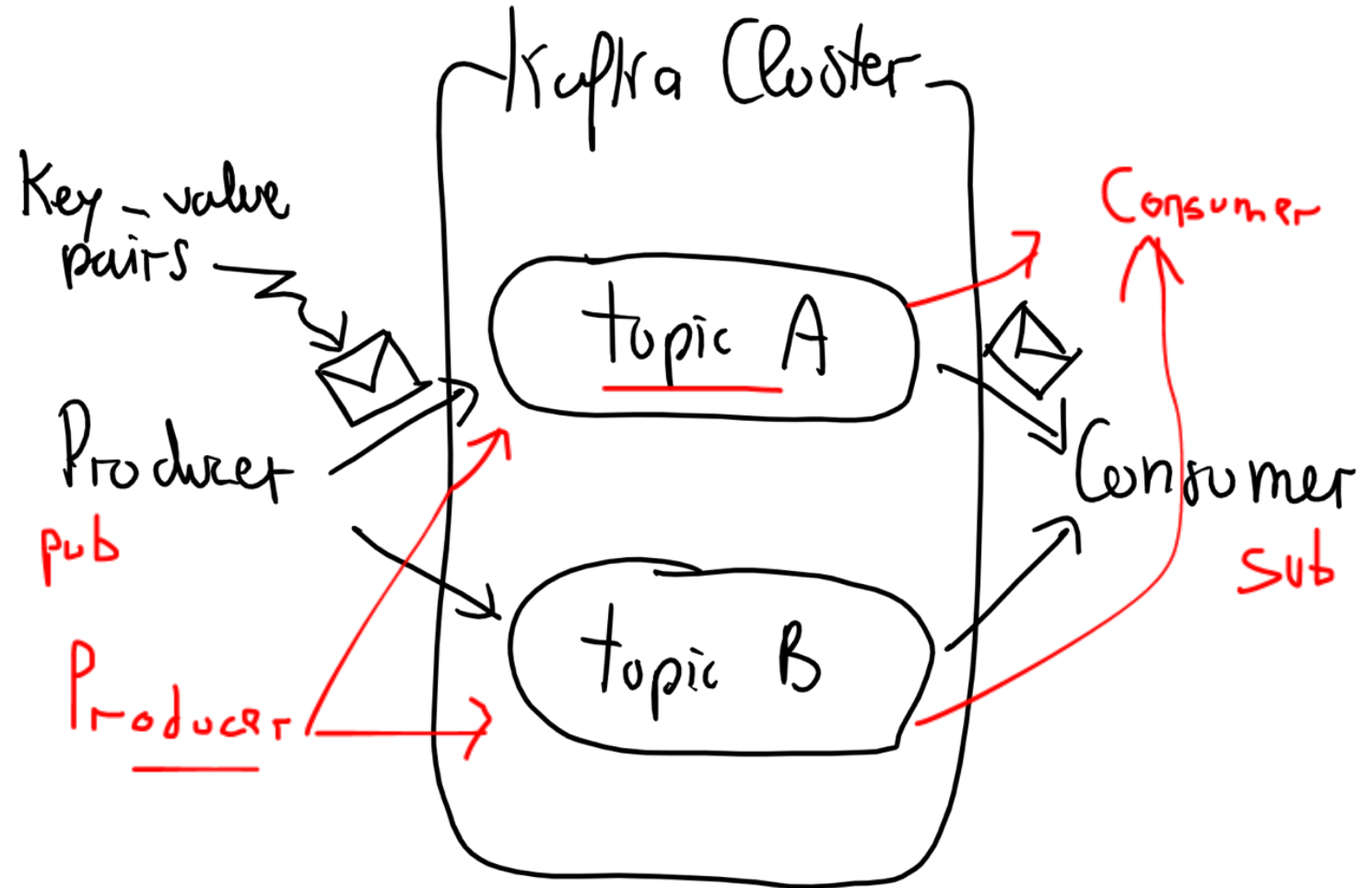


Kafka History

- In 2010, it was created at LinkedIn to
 - Simplifying data pipelines
 - Processing streaming data for batch and real-time analytics
 - In 2012, it became a top-level Apache project
 - Now, it is now at the core of LinkedIn's architecture
 - Performs extremely well at very large scale
 - Produces over 2 *trillion* messages per day
- and it is in use at many organizations
- Twitter, Netflix, Goldman Sachs, Hotels.com, IBM, Spotify, Uber, Square, Cisco...

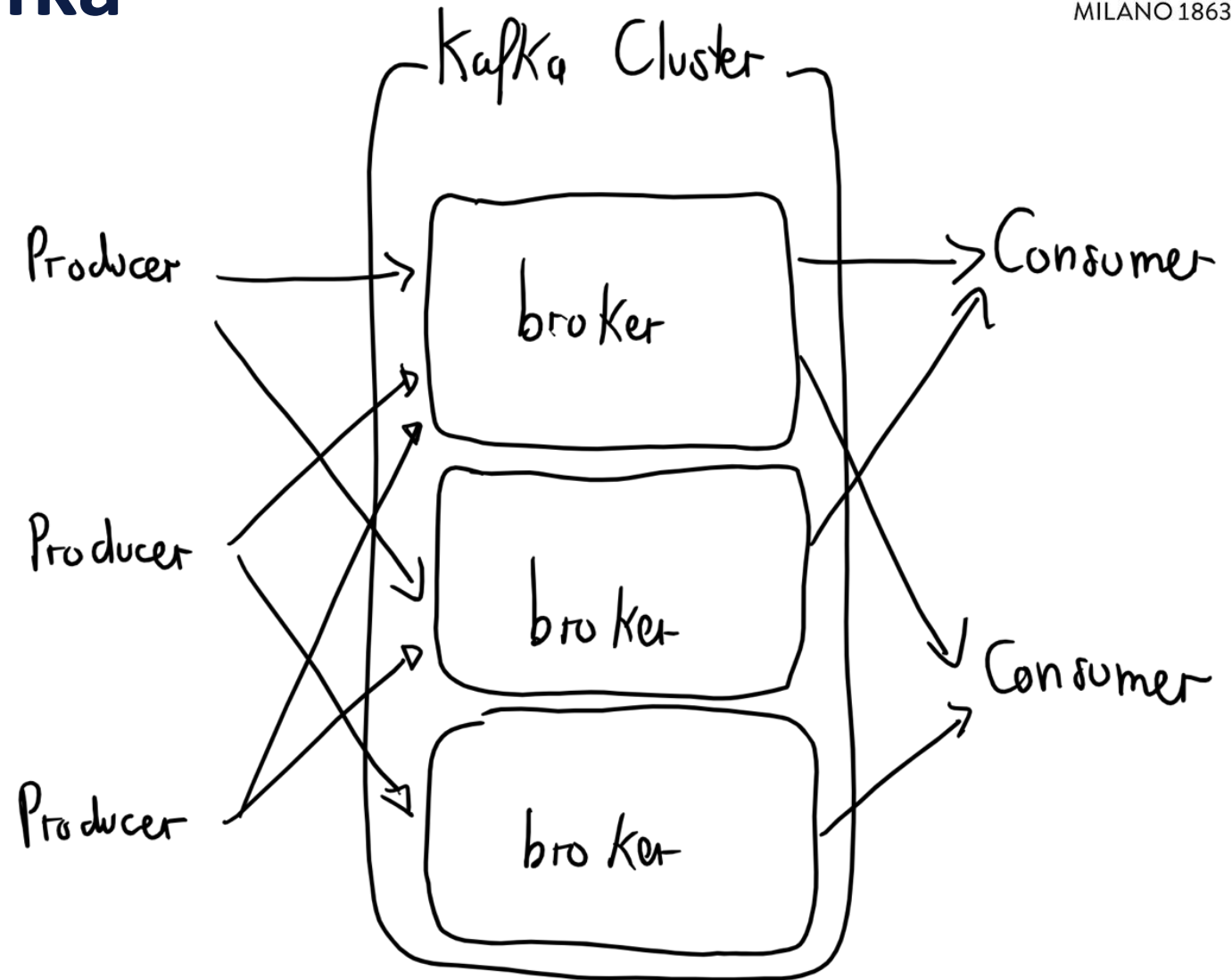
A Conceptual View of Kafka

- **Producers** send messages on topics
 - Publishers in pub-sub terms
- **Consumers** read messages from topics
 - Subscribers in pub-sub terms
- **Topics** are streams of messages
- **A message** is a key-value pair + metadata
- A Kafka cluster manages topics
 - No content-based routing



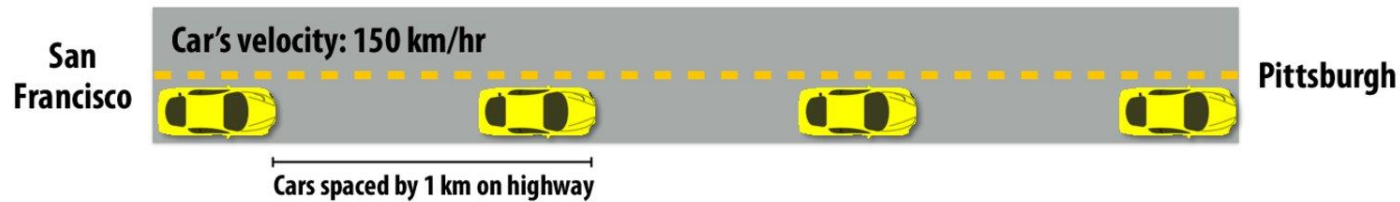
A System View of Kafka

- **Brokers** are the main storage and messaging components of the Kafka cluster
- Producers publish messages on brokers
- Consumers subscribe to brokers to receive messages



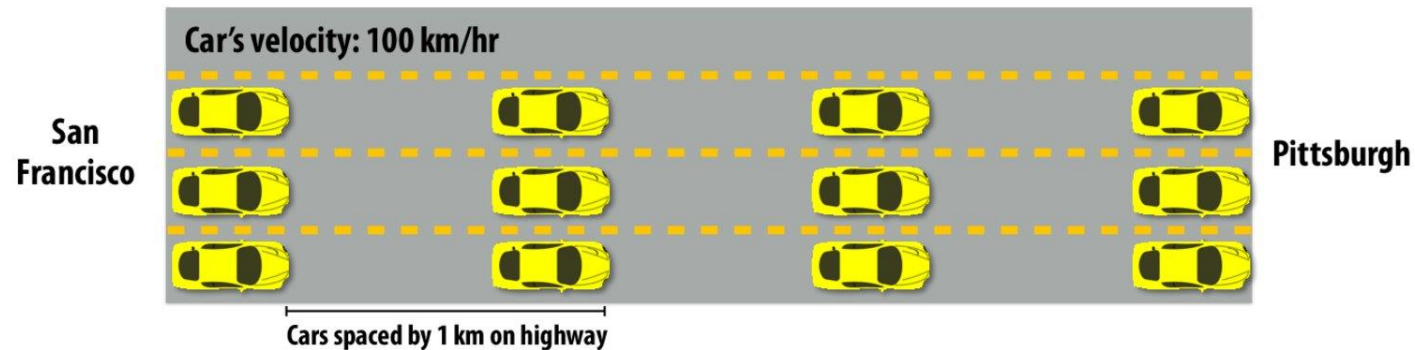
Notice the similarity

Improving throughput



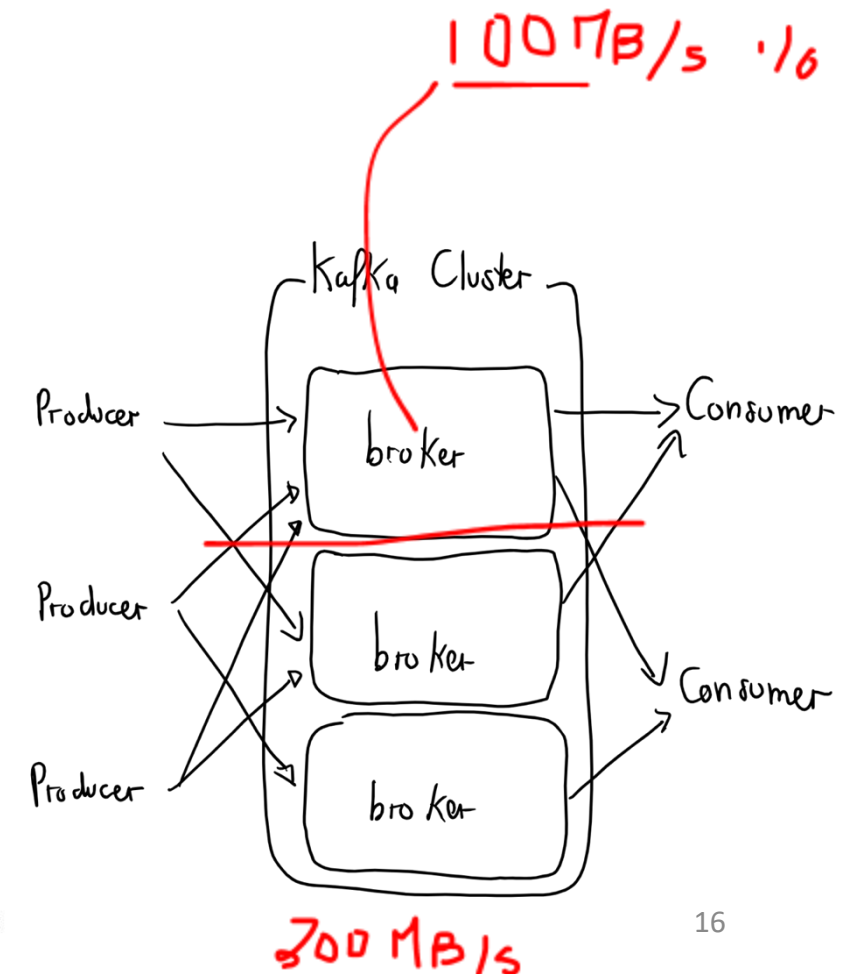
Approach 1: drive faster!

Throughput = 150 people per hour (1 car every 1/150 of an hour)



Approach 2: build more lanes!

Throughput: 300 people per hour (3 cars every 1/100 of an hour)





More on brokers

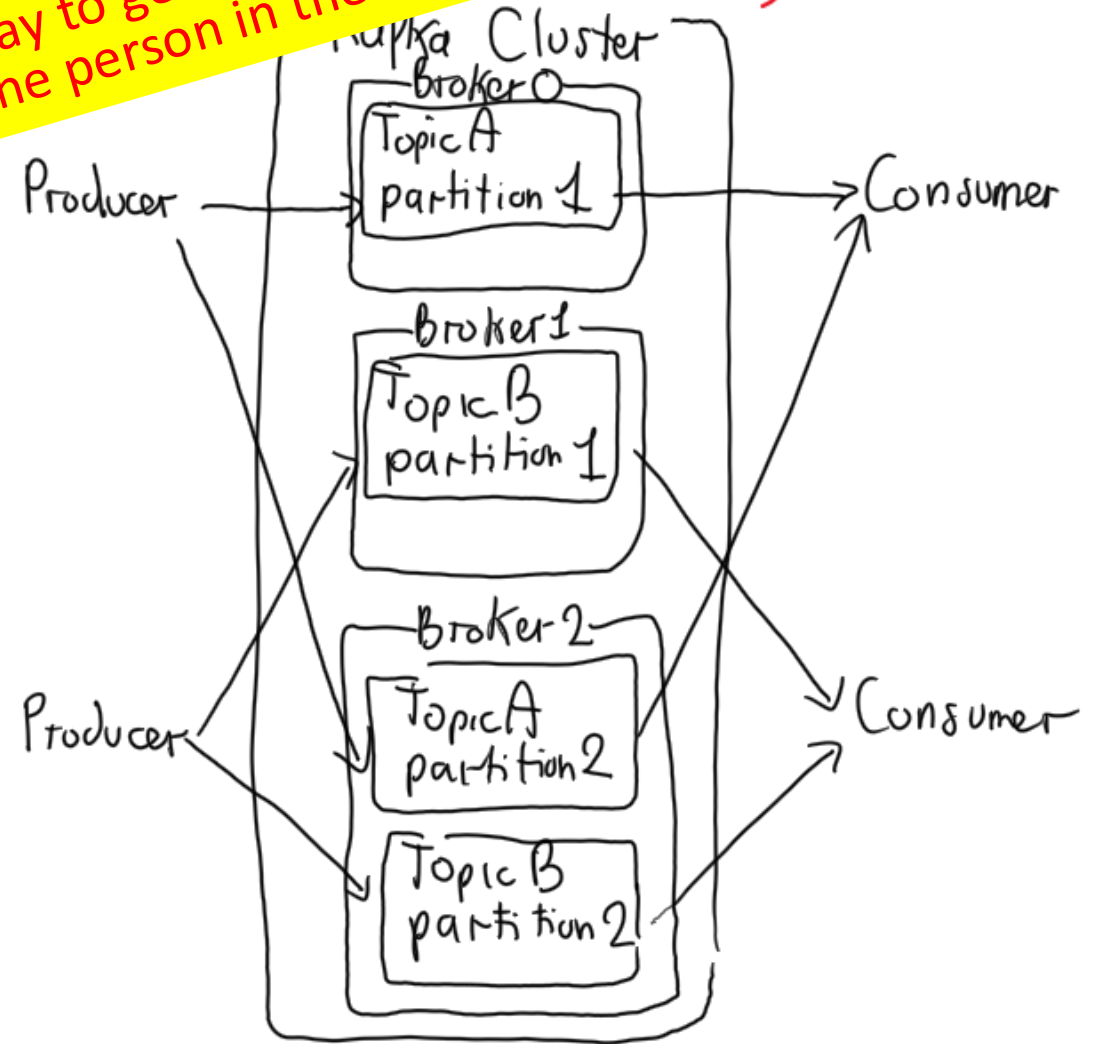
- Brokers receive and store messages when they are sent by the Producers
- A production Kafka cluster will have three or more Brokers
 - Each can handle hundreds of thousands, or millions, of messages per second
- Typically, a Broker manages multiple Partitions

Reconciling the two views

- Topics are **partitioned** across brokers
- **Producers shard messages** over the partitions of a certain topic
- Typically, hash partitioning, based on the message's key, determines the partition a message is assigned to
 - If the key is null, then round-robin partitioning is adopted

$K \rightarrow \{0, 1, 2\}$ $K_1 \rightarrow |Z| \rightarrow 1$ $1 \rightarrow 1$ $2 \rightarrow 0$

The message is the car
The key is a way to get assigned to the lane
The value is the person in the car



A Physical View of topic partition

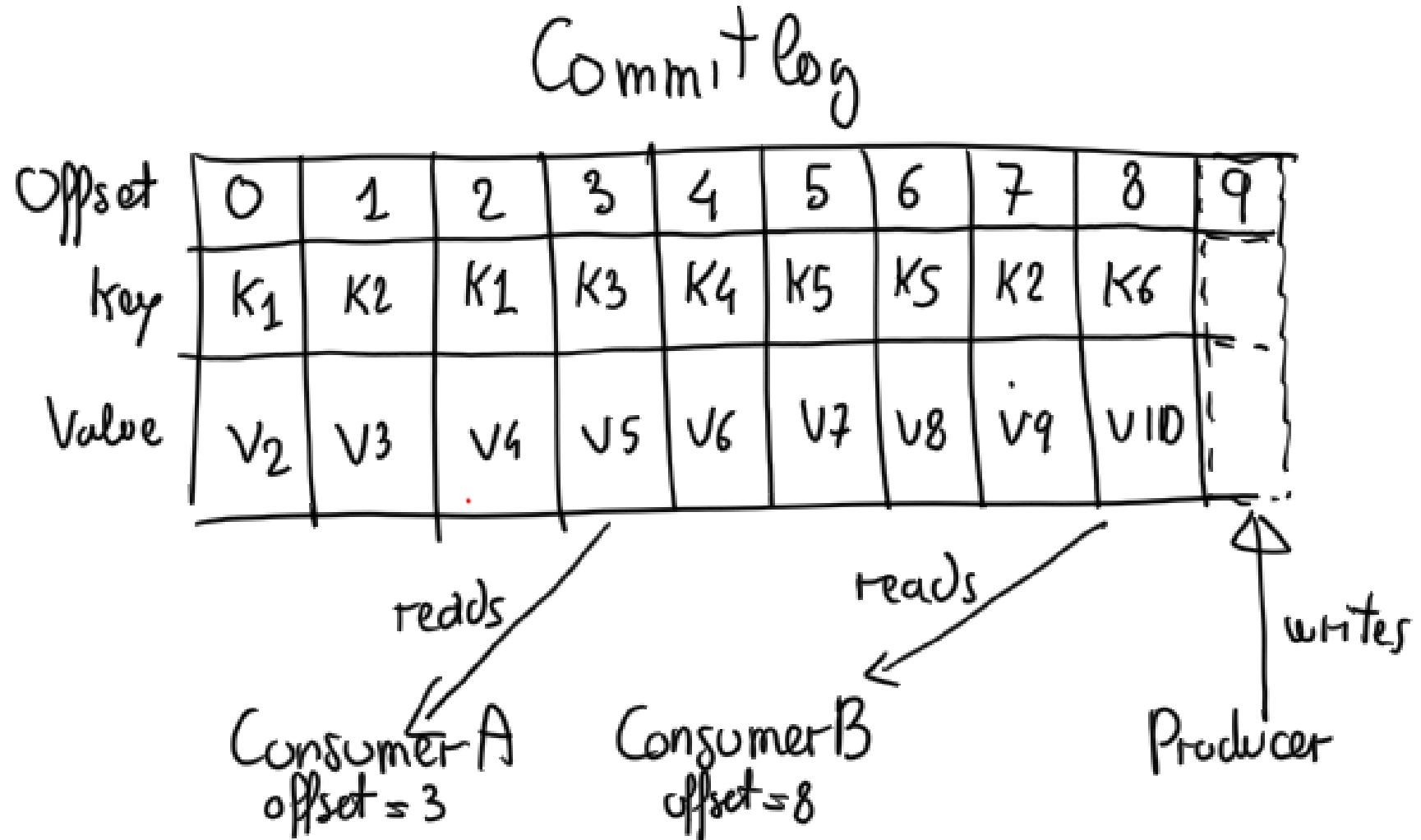
- Each Partition is stored on the Broker's disk as one or more log files
- Each message in the log is identified by its **offset** number

Commit log

Offset	0	1	2	3	4	5	6	7	8	
key	K ₁	K ₂	K ₁	K ₃	K ₄	K ₅	K ₅	K ₂	K ₆	
Value	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈	V ₉	V ₁₀	

A Physical View of topic partition (cont.)

- Messages are always appended
- Consumers can consume from different offset
- Brokers are single thread to guarantee consistency





Log Retention

- Duration default: messages will be retained for seven days
- Duration is configurable per Broker by setting
 - a time period
 - a size limit (number of messages or size in GB)
- The topic configuration can override a Broker's retention policy
- When cleaning up a log
 - the **default** policy is to **delete**
 - An **alternative** policy is **compact**

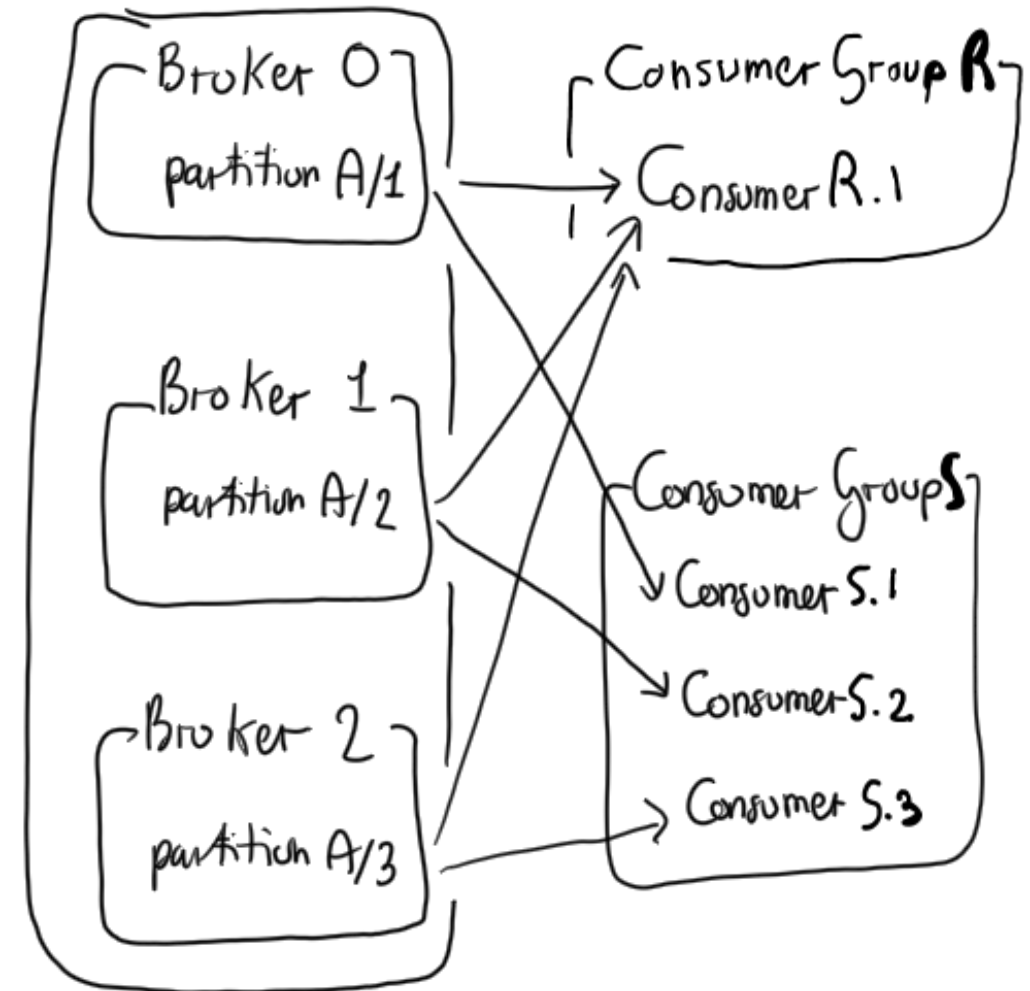
Log compaction

A compacted log retains at least the last known message value for each key within the Partition

Before compaction										After compaction						
Offset key Value	0	1	2	3	4	5	6	7	8	0	1	2	3	4	5	
	K ₁	K ₂	K ₁	K ₃	K ₄	K ₅	K ₅	K ₂	K ₆	K ₂	K ₃	K ₄	K ₅	K ₂	K ₆	
	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇	V ₈	V ₉	V ₁₀	V ₄	V ₅	V ₆	V ₈	V ₉	V ₁₀	

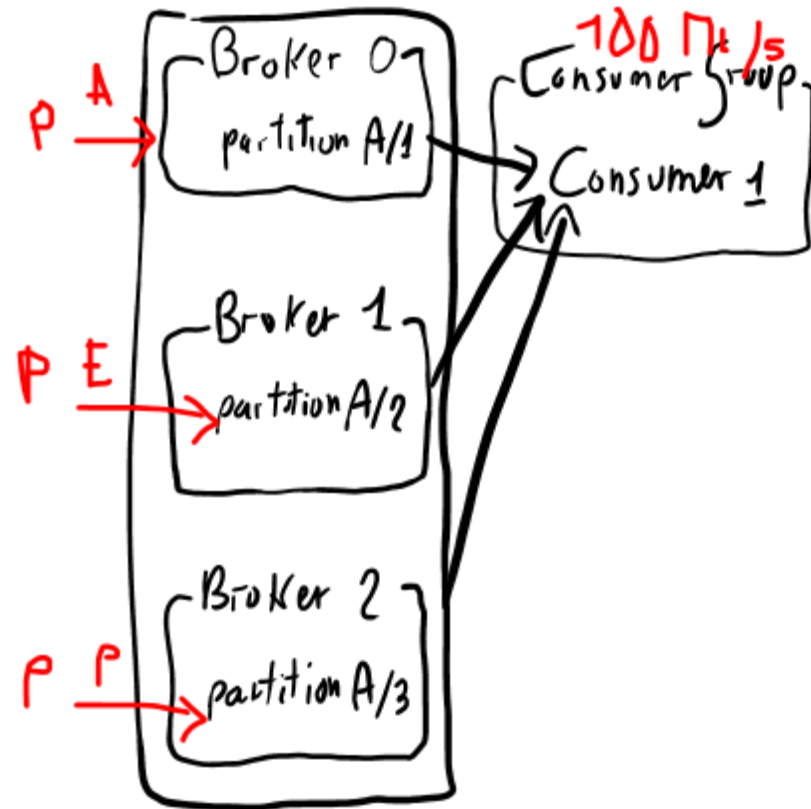
Topic partitioning invites distributed consumption

- Different Consumers can read data from the same Topic
 - By default, each Consumer will receive all the messages in the Topic
- Multiple Consumers can be *logically* combined into a **Consumer Group**
 - Consumer Groups provide scaling capabilities
 - Each Consumer is assigned a subset of Partitions for consumption

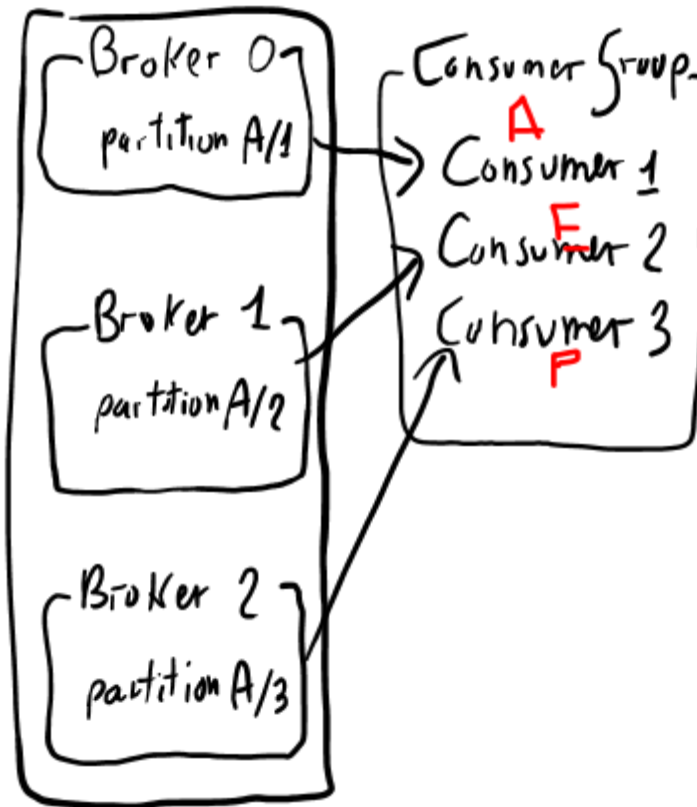


Consumer Group and scalability

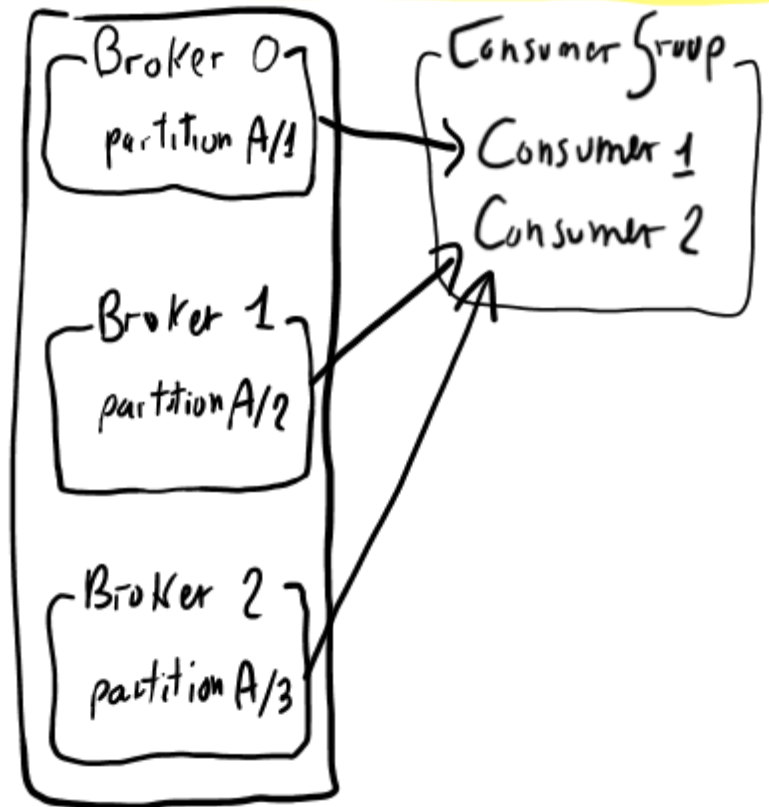
t_0 initial



t_1 scaling out



t_2 scaling in

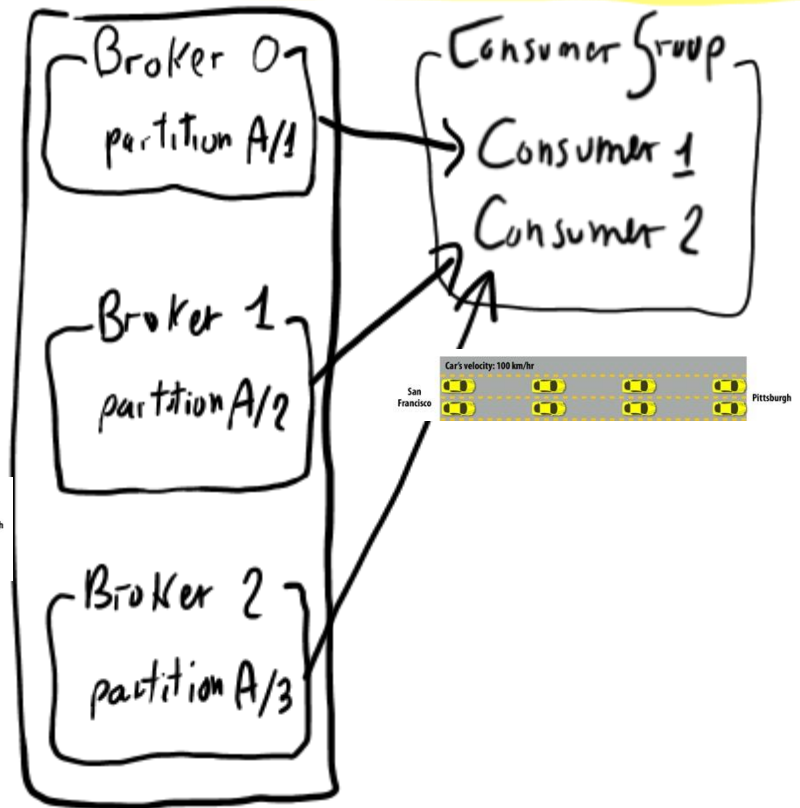
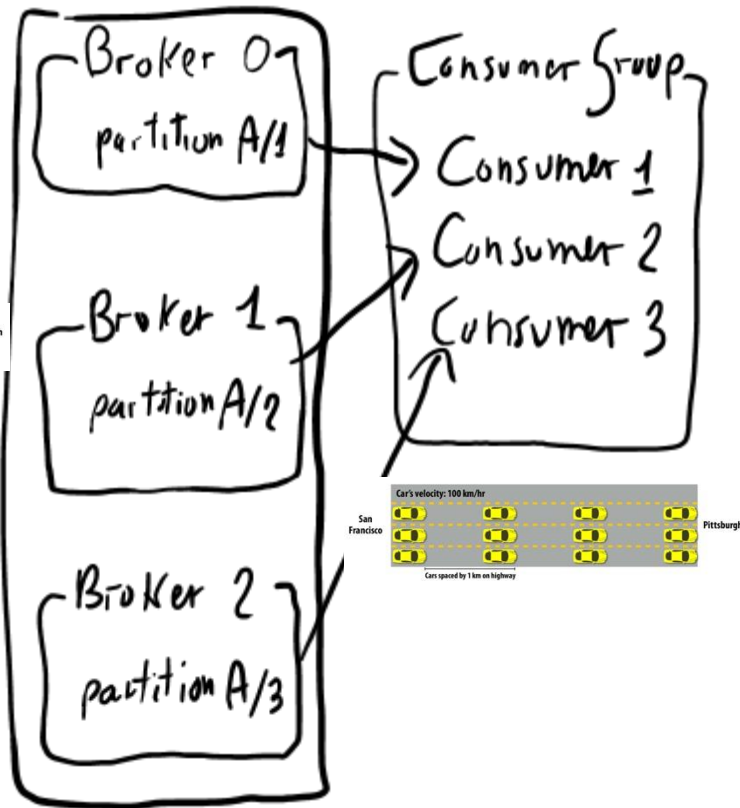
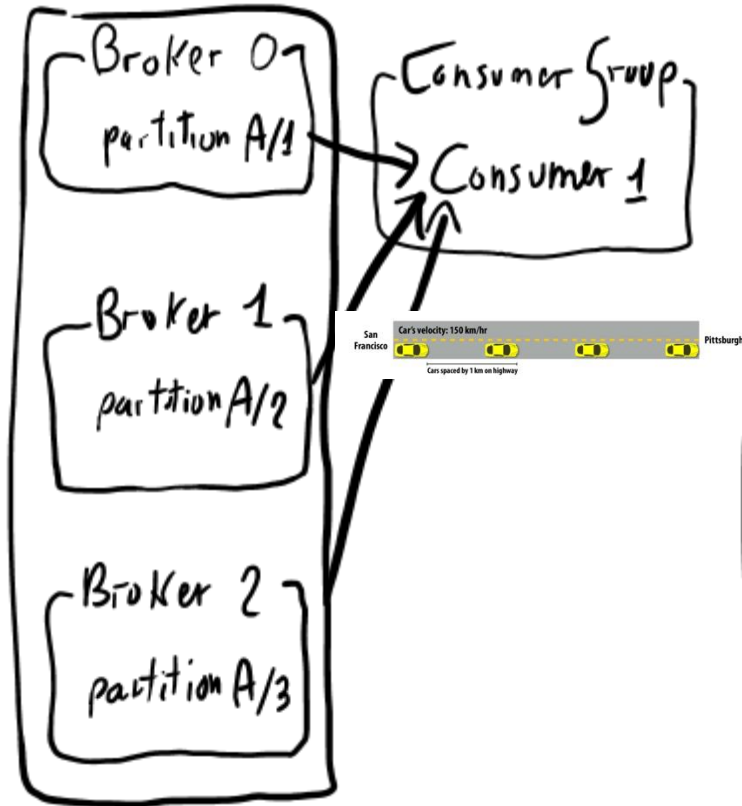


Once again, notice the similarity

t_0 initial

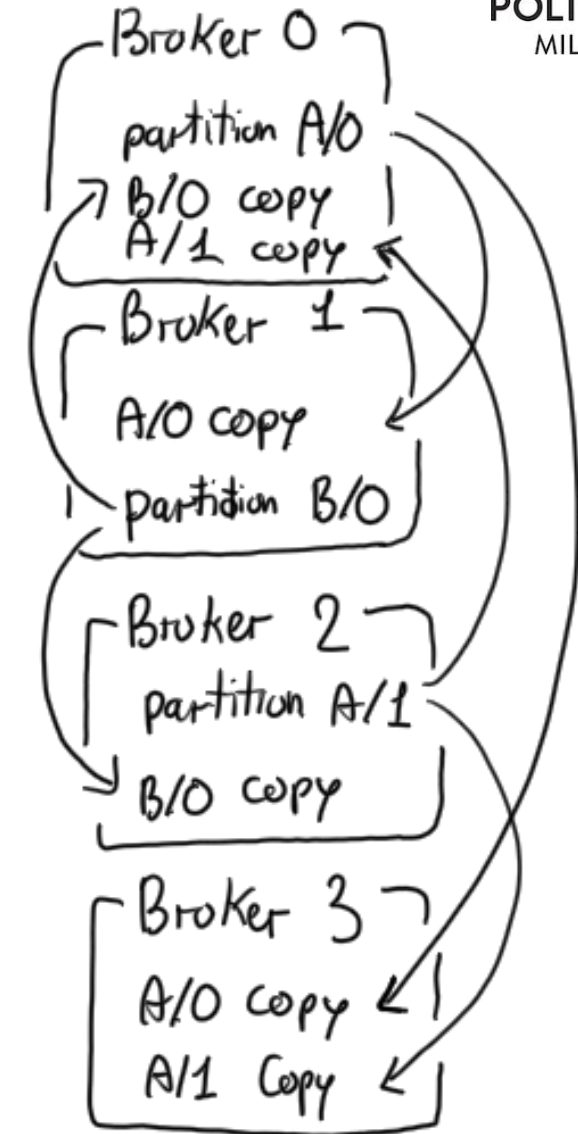
t_1 scaling out

t_2 scaling in



Fault Tolerance via a Replicated Log

- Kafka maintains replicas of each partition on other Brokers in the cluster
 - The number of replicas is configurable
- One Broker is the **leader** for that Partition
 - All writes and reads go to and from the leader
 - Other Brokers are **followers**
- Replication provides fault tolerance in case a Broker goes down

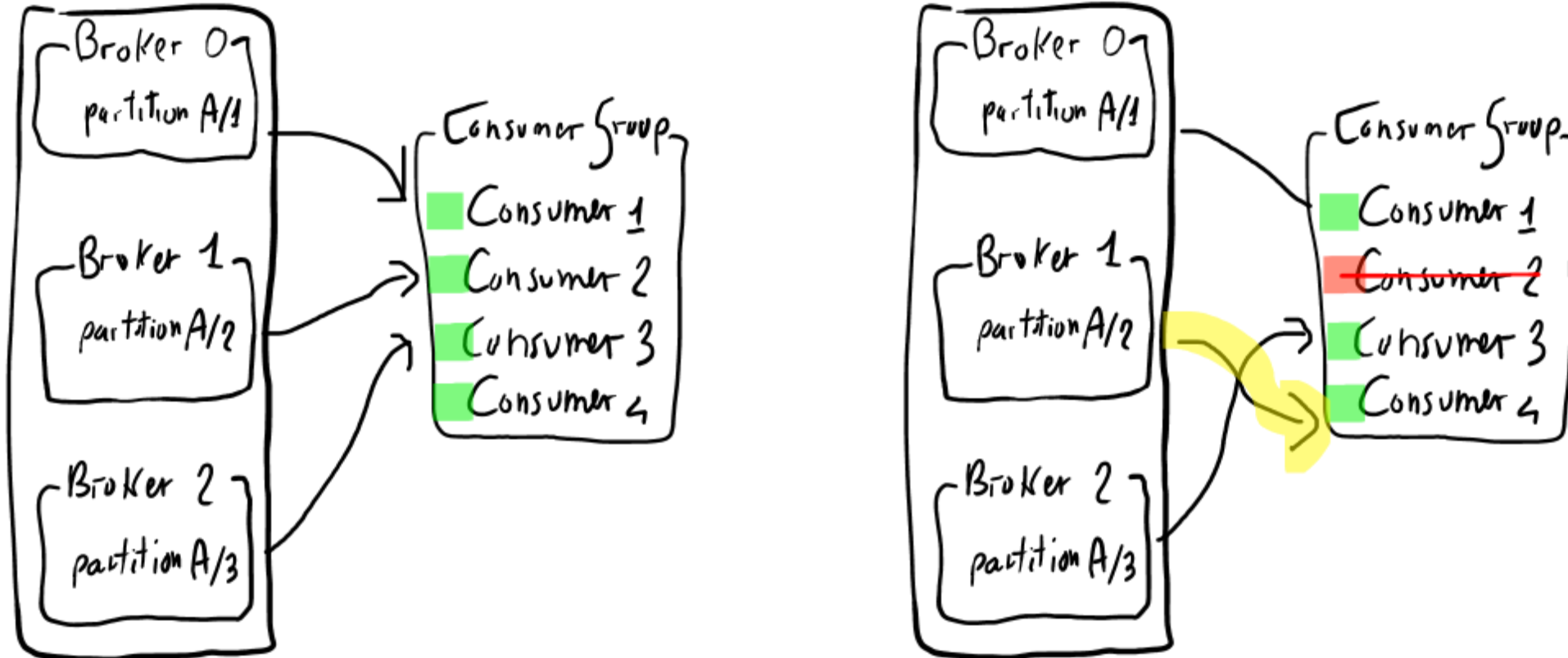




Producers, fault tolerance and durability

- Producers can control durability by requiring the leader to make a number of acknowledgments before considering the request complete.
 - **acks=0**
The producer will not wait for any acknowledgment from the broker
 - **acks=1**
The producer will wait until the leader has written the record to its local log
 - **acks=all**
The producer will wait until all in-sync replicas have acknowledged receipt of the record
- For more information <https://kafka.apache.org/documentation.html#semantics>

Consumer Group and fault tolerance





Quiz

- Provide the correct relationship — 1:1, 1:N, N:1, or N:N —
 - Broker to Partition — ?
 - Key to Partition — ?
 - Producer to Topic — ?
 - Consumer Group to Topic — ?
 - Consumer (in a Consumer Group) to Partition — ?



POLITECNICO
MILANO 1863

Kafka

Scaling continuous ingestion

Emanuele Della Valle

emanuele.dellavalle@polimi.it